

Tolerância a Faltas

Relatório Trabalho Prático

Francisco Macedo Ferreira
PG55942

Ivan Sérgio Rocha Ribeiro
PG55950

Diogo Alexandre Correia Marques
PG?????

Neste trabalho prático, pretendemos identificar e analisar possíveis vulnerabilidades do algoritmo Raft perante a falhas assertivas que comprometam as propriedades de segurança do mesmo.

```
maelstrom.jar test -w lin-kv --bin .\target\release\pessi-raft.exe --time-limit 60 --  
node-count 3 --concurrency 10n --rate 100 --nemesis partition --nemesis-interval 3
```

A. Falsificação/Adulteração de Mensagens

Identificação da Falta

Um nodo malicioso pode falsificar ou adulterar (via *man-in-the-middle*) mensagens de outros nodos.

Impacto

O impacto é total, um nodo malicioso pode tomar controlo total do cluster de nodos. Um nodo malicioso pode:

- fazer com que eleições sejam derrubadas, falsificando mensagens de votação;
- **A ADICIONAR MAIS**

Demonstração do Impacto

A demonstração de falsificação via Maelstrom é simples, visto que é possível colocar qualquer "src" na mensagem e o Maelstrom não valida a origem da mensagem. Fora do Maelstrom, num sistema com comunicação via IP, também é possível falsificar mensagens, por exemplo, com recurso a *IP Spoofing*.

A FAZER

Medidas de Mitigação

Seria possível mitigar completamente este problema adicionando autenticação às mensagens, por exemplo, através de assinaturas digitais.

Demonstração das Medidas de Mitigação

Devido à natureza do problema, não relacionado com os conteúdos abordados na unidade curricular, não iremos implementar a mitigação.

B. Voto Duplo

Identificação da Falta

Um nodo malicioso pode votar em dois candidatos diferentes numa eleição.

Impacto

Permite que dois nodos sejam eleitos como líder, o que quebra a propriedade de **Election Safety** do algoritmo.

Demonstração do Impacto

A FAZER

Medidas de Mitigação

É possível resolver este problema com uma mensagem adicional, por exemplo, *ElectedBy*, que é enviada num fim de eleição, por todos os nodos que foram eleitos para todos os outros nodos, com a lista de nodos que votaram nele. Caso seja detetado um nodo que votou em dois candidatos diferentes, outros nodos podem ignorar o seu voto futuramente (*black-listed*).

Daqui surge um novo problema, que é, um nodo bizantino pode dizer que recebeu votos de quem não votou nele, o que faria que esse outro nodo fosse *black-listed* indevidamente. Uma solução possível a este problema é recorrer a assinaturas digitais, que permitiriam identificar a autenticidade do voto.

Demonstração das Medidas de Mitigação

A FAZER (sem criptografia)

C. Negação de Serviço por Spam de Eleições

Identificação da Falta

Cada *follower* tem uma *timer* para dar *timeout* e começar uma eleição, quando não recebe atualizações de um líder. Um nodo malicioso pode simplesmente ignorar esse *timer* e começar sempre uma nova eleição. A cada momento desses, o nodo malicioso, incrementa o seu *term* e tenta eleger-se como líder. Como o seu *term* é maior que o do líder, todos os nodos (incluindo o líder) votam nele, e ele passa a ser o líder.

Este nodo malicioso pode continuar a fazer isto indefinidamente, sempre que um líder é eleito.

Impacto

Como não há um líder estável, a propriedade de *liveness* não é respeitada, isto é, o sistema não consegue fazer progresso, visto que não há tempo suficiente para replicar as entradas no log.

Demonstração do Impacto

A FAZER

Medidas de Mitigação

Para este problema, não há uma solução concreta, e teremos que recorrer a soluções heurísticas.

Uma solução possível é ignorar começos de eleições demasiado cedo, por exemplo, se um *term* começou há menos de 1 minuto, ignorar o começo dele. No entanto, esta solução pode fazer com que o sistema volte demasiado tempo a voltar ao normal caso o líder realmente morra nesse espaço de tempo.

Outra solução possível é limitar o número de eleições que um nodo pode iniciar num dado espaço de tempo, por exemplo, 1 eleição por hora.

Demonstração das Medidas de Mitigação

A FAZER

D. Enviar mensagens de log sem ser o líder

Identificação da Falta

Um nodo malicioso pode enviar mensagens de *log* (*AppendEntries*) sem ser o líder. Na implementação atual, sempre que um nodo recebe uma mensagem de *log*, verifica se o seu

term é maior ou igual ao term da mensagem, e se for, declara imediatamente o nodo emissor como líder, caso ainda não seja.

Impacto

Qualquer nodo pode tornar-se líder, mesmo não tendo recebido votações positivas de outros nodos.

Demonstração do Impacto

A FAZER

Medidas de Mitigação

Uma possível solução para este problema, será a mesma do que a solução para o problema do voto duplo, que é adicionar uma mensagem adicional, por exemplo, *ElectedBy*, que é enviada num fim de eleição, por todos os nodos que foram eleitos para todos os outros nodos, com a lista de nodos que votaram nele. Caso os outros nodos detectarem que o líder malicioso não foi eleito por ele, podem começar uma nova eleição e bloquear o líder malicioso.

Como um líder malicioso teria que enviar um quorum de nodos no *ElectedBy* para este ser válido, um quorum de nodos não votou nele, e portanto um quorum de nodos o bloquearia, impedindo que ele se torne líder futuramente.

Demonstração das Medidas de Mitigação

A FAZER

E. *Commit* de log sem ser aceite pela maioria

Identificação da Falta

Um nodo líder malicioso pode incrementar o `commit_index` sem ter recebido confirmação da maioria dos nodos.

Impacto

Os *logs* podem não estar atualizados numa maioria dos nodos, o que faria com que caso o líder falhasse, dados seriam perdidos. Quebra o princípio de *safety*.

Demonstração do Impacto

A FAZER

Medidas de Mitigação

É possível mitigar este problema recorrendo a assinaturas digitais, que permitiriam identificar a autenticidade dos votos de outros nodos.

Demonstração das Medidas de Mitigação

Da mesma forma que na solução da Falta A, devido à natureza do problema, que não está relacionada com os conteúdos abordados na unidade curricular, não iremos implementar a mitigação.